

PucePark — Análisis de Diseño de Sistemas de Información

Proyecto Integrador P02 · PUCE TEC Sistema de gestión de parqueaderos universitarios (app móvil + backend de microservicios).

1. Requerimientos Funcionales (RF)

ID	Requerimiento	Actor	Prioridad
RF-01	Iniciar sesión con una única pantalla que diferencia estudiante/guardia según el rol del token (AWS Cognito).	Estudiante, Guardia	Alta
RF-02	Un usuario autenticado que no tenga cuenta en el sistema ve el mensaje “No estás registrado, ve a Secretaría”.	Estudiante	Media
RF-03	Completar el perfil (nombre y apellido, placa, permiso / cédula) mediante onboarding antes de usar el sistema.	Estudiante, Guardia	Alta
RF-04	Listar las zonas de parqueo con su disponibilidad (disponibles/ocupados).	Estudiante, Guardia	Alta
RF-05	Ver el mapa de puestos de una zona con estados por color (verde=disponible, amarillo=mi puesto, rojo=ocupado).	Estudiante, Guardia	Alta
RF-06	Ocupar un puesto disponible. El sistema impide ocupar más de uno a la vez.	Estudiante	Alta
RF-07	Liberar el puesto propio desde un botón fijo, sin desplazarse por la lista.	Estudiante	Alta
RF-08	Guardia: registrar entrada manual (forzar ocupación) indicando la placa del vehículo.	Guardia	Media
RF-09	Guardia: forzar la liberación de un puesto ocupado.	Guardia	Media
RF-10	Consultar el historial de parqueos del propio usuario.	Estudiante	Media
RF-11	Consultar estadísticas personales mensuales (horas, sesiones, racha).	Estudiante	Baja
RF-12	Consultar el ranking mensual de usuarios por horas acumuladas.	Estudiante, Guardia	Baja
RF-13	Editar el perfil (ícono de lápiz) y ver los datos personales.	Estudiante, Guardia	Media
RF-14	Administrar zonas y puestos (crear, actualizar, eliminar) desde un panel web de administración (servido por nginx en /admin/).	Admin	Media
RF-15	Restringir acciones según rol; un usuario sin permiso recibe 401/403.	Sistema	Alta
RF-16	Cerrar sesión y volver a iniciar sin residuos de sesión previa.	Estudiante, Guardia	Media

2. Requerimientos No Funcionales (RNF)

ID	Requerimiento	Cómo se cumple en PucePark
RNF-01 Seguridad	Autenticación y autorización basada en tokens.	JWT emitido por AWS Cognito; cada microservicio valida el token contra el JWKS del <i>issuer</i> (Resource Server). Roles por <code>cognito:groups</code> .
RNF-02 Escalabilidad	El sistema debe poder crecer por servicio.	Microservicios independientes (park-app, users-service) con base de datos por servicio ; servicios stateless detrás de nginx → replicables horizontalmente.
RNF-03 Disponibilidad	Recuperación ante fallos de contenedor.	<code>restart: unless-stopped</code> + <code>healthcheck</code> de las BDs en docker-compose; <code>depends_on: condition: service_healthy</code> .
RNF-04 Concurrencia	Evitar doble ocupación del mismo puesto.	Bloqueo pesimista (<code>findByIdWithPessimisticLock</code>) al ocupar.
RNF-05 Mantenibilidad	Código organizado y comprensible.	Arquitectura en capas (controller/service/repository) + DTOs/mappers/entities; una excepción por archivo + manejador global.
RNF-06 Portabilidad	Despliegue reproducible en cualquier entorno.	Todo dockerizado; punto de entrada único por nginx (puerto 80).
RNF-07 Usabilidad	Interfaz clara con validaciones.	App iOS con validación de formularios (placa, permiso, nombre y apellido), leyenda de colores, estados de carga/error.
RNF-08 Consistencia	Respuestas de error uniformes.	<code>GlobalExceptionHandler</code> con <code>ExceptionResponse { message, source }</code> y códigos HTTP adecuados (404/409/400/401/403).

3. Tabla de Casos de Uso

Caso de uso	Actor	Precondición	Flujo principal	Postcondición
CU-01 Iniciar sesión	Estudiante/Guardia	Tener credenciales PUCE	1) Ingresa usuario/clave 2) Cognito valida y emite JWT 3) La app lee el rol y navega a Zonas	Sesión activa con token y rol
CU-02 Completar perfil	Estudiante/Guardia	Sesión activa, perfil incompleto	1) Se muestra onboarding 2) Ingresa nombre+apellido, placa/cédula, permiso 3) Guarda	Perfil <code>complete=true</code> en users-service
CU-03 Ocupar puesto	Estudiante	Perfil completo, sin puesto activo	1) Elige zona 2) Toca un puesto disponible 3) Confirma "Ocupar"	Puesto en OCUPADO; registro en historial
CU-04 Liberar puesto	Estudiante	Tener un puesto activo	1) Toca "Liberar" 2) El sistema cierra la sesión de parqueo	Puesto en DISPONIBLE; <code>exit_date</code> registrado
CU-05 Forzar ocupación	Guardia	Rol GUARD, puesto disponible	1) Selecciona puesto 2) Ingresa placa 3) Registra entrada	Puesto OCUPADO a nombre "GUARDIA:usuario"
CU-06 Forzar liberación	Guardia	Rol GUARD, puesto ocupado	1) Selecciona puesto 2) Confirma liberación	Puesto DISPONIBLE
CU-07 Ver ranking mensual	Todos	Sesión activa	1) Abre pestaña Ranking 2) Selecciona mes	Lista ordenada por horas
CU-08 Administrar zonas	Admin	Rol ADMIN	CRUD de zonas/puestos	Catálogo actualizado

4. GitFlow en GitHub (criterio 1.2)

El equipo gestiona el código en **GitHub** con un flujo basado en ramas + Pull Requests:

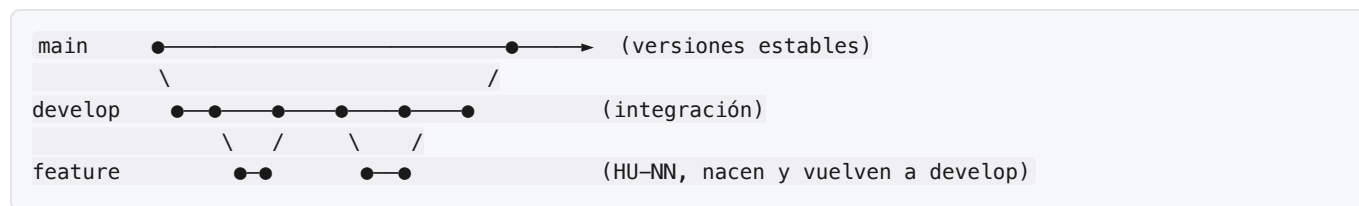
Repositorios: - Backend (microservicios): `github.com/Thyago23/ae_2026_01_Taco_Cede-o_1462_PucePark` - Frontend (app iOS): `github.com/BryanTaco/PuceParkFront`

Se aplica el modelo **GitFlow clásico** (Vincent Driessen), el mismo enseñado en clase con el proyecto de referencia *Micromercado*.

Ramas de larga vida: - **main** — código en producción; solo recibe versiones estables (merges de `release/*` o `hotfix/*`). - **develop** — rama de integración; acumula las funcionalidades terminadas antes de una versión.

Ramas de apoyo (temporales): - **feature/HU-NN-descripcion** — una por Historia de Usuario; nace de `develop` y regresa a `develop` (ej. `feature/HU-06-ocupar-puesto`). *Convención tomada del ejemplo Micromercado* (`feature/scrum-XX-HU-YY-...`). - **release/x.y** — preparación de una versión; nace de `develop`, se prueba y se mergea a `main` y a `develop`. - **hotfix/x.y.z** — corrección urgente en producción; nace de `main` y regresa a `main` y a `develop`.

Flujo de una Historia de Usuario: 1. `git checkout develop && git pull` 2. `git checkout -b feature/HU-NN-descripcion` 3. Desarrollo + *commits* con prefijos convencionales (`feat:`, `fix:`, `refactor:`). 4. `git push -u origin feature/HU-NN-descripcion` 5. **Pull Request** hacia `develop`, revisión del equipo y *merge*. 6. Al completar el alcance, `release/x.y` desde `develop` → `main` (despliegue).



Convención de commits: mensajes en español con prefijo de tipo (`feat:`, `fix:`, `refactor:`, `docs:`) y co-autoría del equipo (`Co-Authored-By`).

Evidencia en GitHub: - Ambos repos tienen las ramas de larga vida `main` y `develop`. - Rama de HU: `feature/HU-01-documentacion-gitflow` → PR hacia `develop` (esta misma documentación). - Refactor a microservicios integrado por PR (perfiles en `users-service`, ranking sin *joins*, nginx con re-resolución). - Historial de *commits* con prefijos convencionales visible en cada repo.

5. Pruebas Unitarias (criterio 1.3)

El backend incluye pruebas de: - **Services** (lógica de negocio): casos válidos e inválidos, uso de mocks (Mockito) para repositorios. - **GlobalExceptionHandler**: cubre la rama del mensaje por defecto de cada excepción (para JaCoCo). - **Controllers** (MockMvc + JWT): endpoints públicos/privados y validación de roles.

Ejecución: `./gradlew test` (y cobertura con JaCoCo).

6. ADR — Architecture Decision Records (criterio 1.4)

ADR-001 · Arquitectura de microservicios con base de datos por servicio

- **Contexto:** el sistema maneja dos dominios distintos: parqueo (zonas/puestos/historial) e identidad de usuario (perfiles).
- **Decisión:** separar en dos microservicios (`park-app`, `users-service`), cada uno con su propia base PostgreSQL (`puce_park`, `puce_micro`).

- **Consecuencias:** (+) despliegue y escalado independiente, aislamiento de fallos; (–) no se pueden hacer *joins* entre servicios → se resuelve con denormalización (ver ADR-004).

ADR-002 · Autenticación centralizada con AWS Cognito (Resource Server)

- **Decisión:** delegar la identidad a un User Pool de Cognito; cada servicio valida el JWT por su cuenta con el JWKS del *issuer*, sin llamadas entre servicios.
- **Consecuencias:** (+) servicios desacoplados, escalable; (+) roles vía `cognito:groups`; (–) dependencia de un proveedor externo.

ADR-003 · nginx como reverse proxy (punto de entrada único)

- **Decisión:** exponer solo nginx en el puerto 80; enruta `/api/*` → park-app y `/users/*` → users-service. Los servicios quedan internos (`expose`).
- **Consecuencias:** (+) un solo punto de entrada, oculta la topología interna; se añadió `resolver 127.0.0.11` para resolver IPs al reconstruir contenedores.

ADR-004 · Denormalización del nombre en el historial para el ranking

- **Contexto:** el ranking (park-app) necesita mostrar el nombre del usuario, que vive en users-service; el patrón prohíbe *joins* entre BDs.
- **Decisión:** el cliente envía el `fullName` al ocupar; park-app lo guarda en `parking_history.display_name`; el ranking lo lee sin *joins*.
- **Consecuencias:** (+) respeta el aislamiento de servicios; (–) dato duplicado (aceptable, es un valor de solo lectura para el ranking).

ADR-005 · Priorización de requerimientos

- **Decisión:** se priorizó primero el núcleo (login, ver zonas, ocupar/liberar) por ser el objetivo principal; luego historial/ranking/perfil; por último administración.
- **Justificación:** entregar valor temprano al usuario final (estudiante) y asegurar los criterios de “objetivo principal” y “conexión a API con BD” antes que funciones secundarias.

PucePark — Computación en la Nube

Proyecto Integrador P02 · PUCE TEC Despliegue del sistema en entornos virtualizados (Docker) con arquitectura multicapa.

1. Conceptos de computación en la nube (criterio 4.1)

Computación en la nube: entrega de recursos de cómputo (servidores, almacenamiento, red, bases de datos) bajo demanda a través de Internet, con pago por uso y aprovisionamiento elástico.

Modelos de servicio: - **IaaS** (Infraestructura como Servicio): máquinas virtuales, redes y almacenamiento. *PucePark se despliega sobre este modelo: contenedores Docker que pueden correr en una VM/instancia IaaS (EC2, Compute Engine, etc.).* - **PaaS:** plataforma gestionada (runtime + BD administrada). - **SaaS:** software final consumido por el usuario. *La app PucePark sería el SaaS resultante.*

Virtualización vs contenedores: - *Virtualización (VM):* cada instancia incluye su propio SO → mayor aislamiento, más peso. - *Contenedores (Docker):* comparten el kernel del host → arranque rápido, menor consumo, ideales para microservicios. **PucePark usa contenedores.**

Escalamiento vertical vs horizontal

Aspecto	Escalamiento Vertical (scale-up)	Escalamiento Horizontal (scale-out)
Qué hace	Añadir más recursos (CPU/RAM) a la misma máquina	Añadir más instancias de la aplicación
Ventajas	Simple, sin cambios de arquitectura; sin sincronización entre nodos	Alta disponibilidad, tolerancia a fallos, crecimiento casi ilimitado, balanceo de carga
Desventajas	Límite físico del hardware; punto único de fallo; downtime al escalar	Requiere servicios <i>stateless</i> y balanceador; mayor complejidad operativa
En PucePark	Subir CPU/RAM al contenedor <code>park-app</code> si una zona recibe mucha carga	Levantar N réplicas de <code>park-app / users-service</code> detrás de nginx (balanceo), porque son <i>stateless</i> (el estado vive en Postgres y en el JWT)

Conclusión aplicada: PucePark está diseñado para **escalamiento horizontal**: los microservicios no guardan estado en memoria (la sesión es el JWT de Cognito y los datos están en la BD), por lo que se pueden replicar y poner detrás de nginx como balanceador. La base de datos escalaría verticalmente o con réplicas de lectura.

2. Virtualización / contenedores (criterio 4.2)

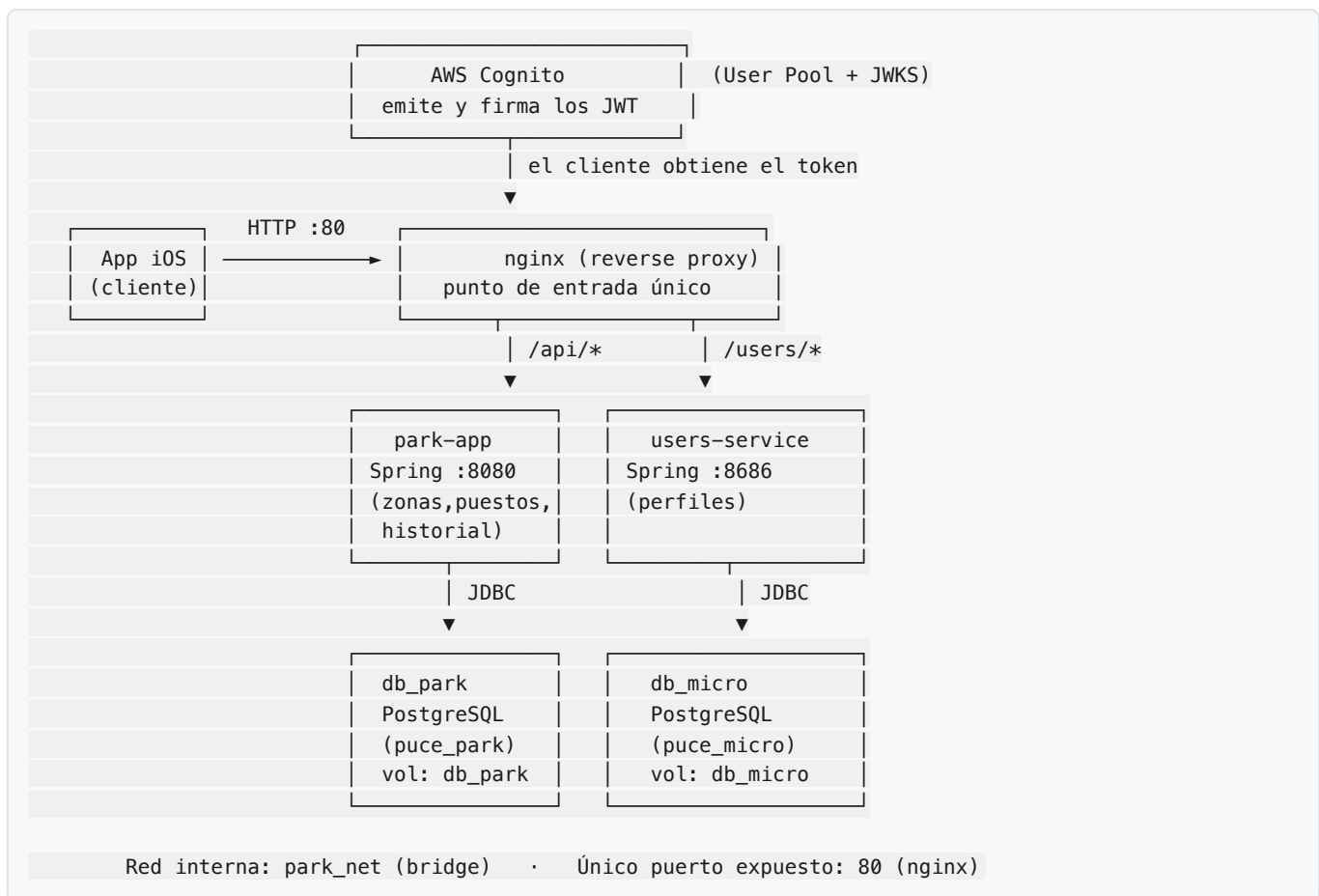
Sistema **multicapa y multiplataforma** orquestado con `docker-compose` (5 contenedores):

Servicio	Imagen/Build	Rol	Puerto
<code>nginx</code>	<code>nginx:alpine</code>	Reverse proxy + panel web <code>/admin/</code> · punto de entrada único	80 → host
<code>park-app</code>	build (Spring Boot/Kotlin)	API de parqueo (<code>/api/*</code>)	8080 (interno)

Servicio	Imagen/Build	Rol	Puerto
users-service	build (Spring Boot/Kotlin)	Microservicio de perfiles (/users/*)	8686 (interno)
db_park	postgres:16-alpine	BD del parqueo (puce_park)	5434 → host
db_micro	postgres:16-alpine	BD de usuarios (puce_micro)	5435 → host

Características de infraestructura implementadas: - **Red privada** park_net (bridge) — los servicios se comunican por nombre de contenedor. - **Volúmenes persistentes** db_park_data, db_micro_data — los datos sobreviven a reinicios. - **Healthchecks** en las BDs (pg_isready) + depends_on: condition: service_healthy. - **Multi-stage build** en el Dockerfile (compila con Gradle y empaqueta solo el JRE + jar). - **Punto de entrada único**: solo nginx publica puerto al host; los servicios usan expose.

3. Diseño de arquitectura de infraestructura (criterio 4.3)



Principios de diseño: - **Aislamiento por servicio**: cada microservicio tiene su BD; no comparten esquema ni hacen *joins* cruzados. - **Sin llamadas entre servicios**: la confianza se delega al JWT de Cognito (cada servicio valida por su cuenta). - **Seguridad perimetral**: solo nginx queda expuesto; las BDs y servicios no son accesibles directamente desde fuera (los puertos 5434/5435 se publican solo para inspección en desarrollo). - **Preparado para la nube**: este docker-compose puede desplegarse en una instancia **IaaS** (AWS EC2 / GCP Compute Engine) tal cual, o migrarse a un orquestador (ECS/Kubernetes) para escalamiento horizontal real replicando park-app / users-service.

4. Despliegue realizado en AWS (evidencia · criterio 4.3)

El sistema **no solo está preparado** para la nube: se desplegó efectivamente en **AWS**.

Backend en EC2 (IaaS)

- Instancia **Amazon EC2** (Amazon Linux 2023, `t3.small`) — modelo **IaaS**.
- Arranque automatizado con **User Data** (`cloud/ec2/user-data.sh`): instala Docker + Compose, clona el repositorio y ejecuta `docker compose up -d --build` (los 5 contenedores).
- **Security Group**: solo `80` (HTTP público) y `22` (SSH restringido a la IP del equipo); las BDs quedan internas.
- Verificación en vivo:
 - `http://<IP-EC2>/api/v1/zonas` → **401** (park-app responde, requiere token).
 - `http://<IP-EC2>/users/me` → **401** (users-service responde).
 - `http://<IP-EC2>/admin/` → **panel web de administración** servido desde la nube.
- Guía completa: `cloud/ec2/README-ec2.md`.

Página de descarga en S3 (almacenamiento en la nube)

- Bucket **Amazon S3** configurado como *static website hosting* con la página de descarga de la app (iOS/APK).
- Guía: `cloud/README-despliegue-s3.md`.

Panel web de administración (cliente adicional en la nube)

El mismo nginx sirve, en `/admin/`, un panel web estático donde el **administrador** inicia sesión con Cognito y gestiona zonas, puestos, ranking e historial — demostrando **un backend, dos clientes** (app iOS + panel web) sobre la misma infraestructura.

5. Ventajas del enfoque para la nube

- **Reproducibilidad**: `docker-compose up -d --build` levanta todo el sistema en cualquier host con Docker (probado en local y en EC2).
- **Elasticidad**: servicios stateless → réplicas horizontales bajo nginx.
- **Costo**: contenedores livianos (alpine) → menor consumo que VMs completas.
- **Portabilidad multiplataforma**: mismas imágenes en local, IaaS o nube pública.

PucePark — Emprendimiento

Proyecto Integrador P02 · PUCE TEC

Cifras del modelo de negocio estimadas para el contexto PUCE (piloto: 1 campus, operación en la nube, equipo reducido). Moneda: USD.

1. Business Model Canvas (criterio 5.1)

PucePark — plataforma inteligente de gestión de parqueaderos universitarios.

Bloque	Contenido
1. Segmentos de clientes	• Universidades e institutos (cliente que paga). • Estudiantes/docentes con vehículo (usuarios). • Personal de seguridad/guardias (usuarios operativos). • Administración de campus.
2. Propuesta de valor	• Encontrar parqueo disponible en tiempo real (menos tiempo dando vueltas). • Control y trazabilidad para guardias (entradas/salidas, placas). • Datos de ocupación para la administración. • Gamificación (ranking) que incentiva buen uso. • Un solo login institucional (Cognito).
3. Canales	• App móvil (iOS; roadmap Android). • Portal web administrativo. • Difusión por la propia universidad (correo institucional, inducción).
4. Relación con clientes	• Autoservicio en la app. • Soporte/SLA a la institución. • Onboarding guiado dentro de la app.
5. Fuentes de ingreso	• Licencia SaaS por campus (suscripción mensual/anual). • Módulos premium (reportes avanzados, integración con talanquera/IoT). • Setup inicial de implementación.
6. Recursos clave	• Equipo de desarrollo. • Infraestructura en la nube (contenedores + Cognito + Postgres). • Código base (2 microservicios + app).
7. Actividades clave	• Desarrollo y mantenimiento. • Operación/monitoreo en la nube. • Soporte y capacitación. • Ventas B2B a instituciones.
8. Socios clave	• Proveedor cloud (AWS/GCP). • Universidad piloto (PUCE). • Proveedores de hardware IoT/talanquera (fase 2).
9. Estructura de costos	• Nube (cómputo, BD, Cognito). • Salarios de desarrollo/soporte. • Licencias y dominios. • Marketing/ventas.

2. Propuesta tecnológica e innovación (criterio 5.2)

Stack: - **App móvil:** SwiftUI (iOS 17+), MVVM, Alamofire, async/await. - **Backend:** Spring Boot 4 + Kotlin, arquitectura de **microservicios** (park-app + users-service), PostgreSQL (BD por servicio). - **Infraestructura:** Docker + docker-compose, **nginx**

como punto de entrada único; preparado para laaS/orquestadores. - **Seguridad:** AWS Cognito (JWT), autorización por roles (estudiante/guardia/admin).

Elementos de innovación y diferenciación: 1. **Disponibilidad en tiempo real** con mapa de puestos por colores (verde/amarillo/rojo). 2. **Arquitectura de microservicios** escalable horizontalmente, lista para crecer a varios campus. 3. **Gamificación** (ranking mensual) para fomentar rotación y buen uso de plazas. 4. **Rol de guardia** integrado (registro manual, forzar liberación) — cubre el proceso real del campus. 5. **Login institucional único** que diferencia el rol automáticamente. 6. **Roadmap IoT:** integración con sensores/talanquera para detección automática (fase 2).

3. Planificación financiera (criterio 5.3)

Supuestos: 1 universidad piloto (PUCE), operación en la nube laaS, equipo pequeño. Moneda: USD.

Inversión inicial (una vez)

Concepto	Estimado
Desarrollo MVP (ya realizado por el equipo)	\$4,000
Configuración cloud + dominios + Cognito	\$300
Diseño de marca / materiales	\$200
Total inicial	\$4,500

Costos operativos mensuales

Concepto	Estimado/mes
Infraestructura cloud (instancia laaS + almacenamiento)	\$60
Base de datos gestionada / respaldo	\$30
AWS Cognito (bajo volumen, capa gratuita + excedente)	\$10
Mantenimiento/soporte (parcial)	\$150
Total mensual	\$250

Ingresos (modelo SaaS B2B)

Concepto	Valor
Licencia por campus (suscripción)	\$400 / mes
Setup inicial de implementación (una vez)	\$500

Proyección simple (1 campus, año 1)

Rubro	Monto
Ingresos año 1 (12 × \$400 + \$500 setup)	\$5,300
Costos operativos año 1 (12 × \$250)	\$3,000

Rubro	Monto
Inversión inicial	\$4,500
Resultado año 1	-\$2,200
Punto de equilibrio	~mes 14-15 (o antes con un 2.º campus)

Análisis: el modelo B2B se vuelve rentable al sumar campus (el costo marginal por campus adicional es bajo gracias a la arquitectura multi-servicio en contenedores). Con 3 campus el resultado del año 1 ya sería positivo (ingresos ≈ \$15,900 vs costos ≈ \$3,000 + \$4,500).